

SIT307

8.1 Distinction Task: Machine Learning Mini Project Sydney Housing Price Prediction and Decision Support System

Ali Alabdouli
224535032
Deakin University

Part 1: Problem Definition and Data Collection

The problem

This project predicts the sale price of a residential property in Sydney from its physical characteristics, location, and sale context. The goal is not accuracy alone, but to understand what actually drives price in a given market, and to flag when a prediction should be trusted versus treated cautiously.

Suburb selection

Three suburbs were chosen to represent genuinely different markets: Mosman (premium), Parramatta (mid-range, transit-oriented), and Campbelltown (affordable, outer Western Sydney). These reflect different buyer profiles and housing stock, not just different price points.

Data collection

104 properties (Mosman: 30, Parramatta: 36, Campbelltown: 38) were collected on 14-15 September 2026 from Domain.com.au sold listings, recording each property's reported sold price (as displayed by Domain, not a formally verified land-title settlement figure), address, sale date, bedrooms, bathrooms, parking, land size, and property type. The full sample spans sale dates from 11 December 2017 to 12 September 2026, unevenly across suburbs (see Part 2). address was retained to support Part 4's investigation but excluded from model training (Part 3).

Two distinct selection-bias mechanisms apply. First, Domain does not disclose a price for every listing; exclusion rates varied by suburb, a systematic sample of 20 Mosman listings during collection found only 2 disclosed a price (10%), versus 18 of 20 Campbelltown listings (90%). Second, within Mosman, disclosed listings skewed toward larger, 3+ bedroom homes, so cheaper Mosman properties are further under-represented.

Land size is missing for 74 of 104 properties (71%), concentrated in strata-title types: Apartment 30/33 (91%), Townhouse 30/37 (81%), Penthouse 4/4 (100%), versus House 6/23 (26%) and Semi-detached 4/7 (57%). Median prices per suburb (Mosman \$3,037,500, Parramatta \$815,500, Campbelltown \$628,797.50) match each suburb's known market tier.

Code:

```
df = pd.read_csv('raw_collected.csv')
df['sale_date'] = pd.to_datetime(df['sale_date'])

print("Shape:", df.shape)
df.head()
```

Output:

Shape: (104, 9)

	address	suburb	price	sale_date	\
0	35/17 Warby Street, Campbelltown	Campbelltown	515000	2026-08-26	
1	66/17 Warby Street, Campbelltown	Campbelltown	522000	2026-08-26	
2	8/30 Broughton Street, Campbelltown	Campbelltown	637595	2026-08-24	
3	5/32 Broughton St, Campbelltown	Campbelltown	620000	2026-08-17	
4	11/39 Chamberlain Street, Campbelltown	Campbelltown	610000	2026-08-14	

	beds	baths	parking	land_size_sqm	property_type
0	2	1	1	NaN	Apartment
1	2	1	1	NaN	Apartment
2	2	1	1	NaN	Townhouse
3	3	1	1	NaN	House
4	2	2	1	NaN	Apartment

Code:

```
print(df['suburb'].value_counts())
print()
print(df.isnull().sum())
```

Output:

```
suburb
Campbelltown    38
Parramatta      36
Mosman          30
Name: count, dtype: int64
```

```
address
0
```

suburb	0
price	0
sale_date	0
beds	0
baths	0
parking	0
land_size_sqm	74
property_type	0
dtype: int64	

Part 2: Data Understanding and Feature Engineering

Price is heavily right-skewed (median \$925K, range \$320K-\$12.5M) and cleanly separated by suburb. The collection window is short and uneven across suburbs (Campbelltown from June 2023, Mosman back to December 2017), so no reliable time trend can be established. A single outlier fence across all three suburbs naturally flags six Mosman properties, since it conflates "expensive suburb" with "unusual sale"; within-suburb fences give a fairer picture, dropping this to two genuine outliers. All six global outliers, and both within-suburb ones, were retained in the modelling data, this analysis is used for interpretation only, not filtering.

Before examining relationships or engineering features, I expected suburb, land size, and property type to matter most. Suburb and property type were confirmed (~5x price spreads), but land size showed essentially zero correlation with price among houses ($r=-0.061$, $n=20$, a small sample that should not be read as land size being generally unimportant); parking emerged as the strongest numeric correlate instead ($r=0.531$). Missing land size was replaced with 0 in `land_size_filled`, paired with a `has_land_size` flag distinguishing "unknown" from "genuinely small." A derived `total_rooms` feature (beds+baths) was created but excluded from modelling, since it is an exact linear combination of two included features.

Code:

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].hist(df['price'], bins=30, color='steelblue', edgecolor='white')
axes[0].set_xlabel('Price ($)')
axes[0].set_ylabel('Count')
axes[0].set_title('Price distribution (raw)')

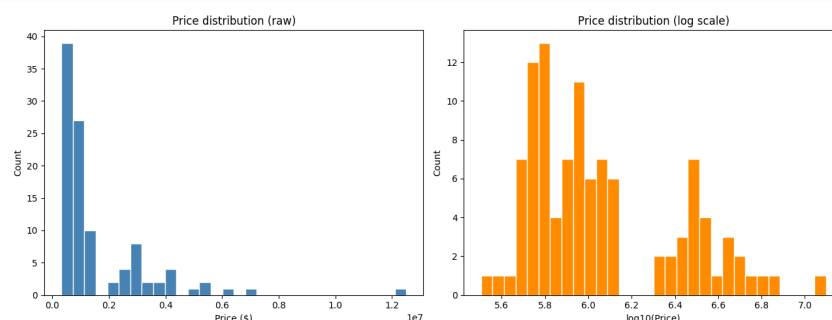
axes[1].hist(np.log10(df['price']), bins=30, color='darkorange', edgecolor='white')
axes[1].set_xlabel('log10(Price)')
axes[1].set_ylabel('Count')
axes[1].set_title('Price distribution (log scale)')

plt.tight_layout()
plt.show()

print(df['price'].describe())
```

Output:

```
count      1.040000e+02
mean       1.639111e+06
std        1.783019e+06
min        3.200000e+05
25%        6.187500e+05
50%        9.250000e+05
75%        2.230000e+06
max        1.250000e+07
Name: price, dtype: float64
```



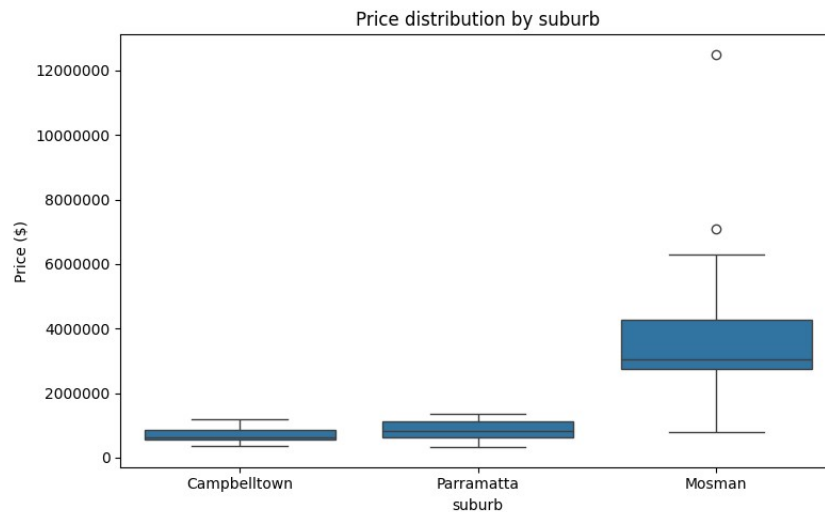
Code:

```
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='suburb', y='price', order=['Campbelltown', 'Parramatta', 'Mosman'])
plt.ylabel('Price ($)')
plt.title('Price distribution by suburb')
plt.ticklabel_format(style='plain', axis='y')
plt.tight_layout()
plt.show()

print(df.groupby('suburb')['price'].describe()[['count', 'mean', '50%', 'min', 'max']])
```

Output:

suburb	count	mean	50%	min	max
Campbelltown	38.0	7.033051e+05	628797.5	363000.0	1185000.0
Mosman	30.0	3.745117e+06	3037500.0	790000.0	12500000.0
Parramatta	36.0	8.719028e+05	815500.0	320000.0	1350000.0



Code:

```
plt.figure(figsize=(10, 6))
colors = {'Campbelltown': 'tab:blue', 'Parramatta': 'tab:orange', 'Mosman': 'tab:green'}
for suburb, group in df.groupby('suburb'):
    plt.scatter(group['sale_date'], group['price'], label=suburb, alpha=0.7,
                color=colors[suburb])
plt.xlabel('Sale date')
plt.ylabel('Price ($)')
plt.yscale('log')
plt.title('Sale price over time by suburb (log scale)')
plt.legend()
plt.tight_layout()
plt.show()
```



Code:

```
Q1, Q3 = df['price'].quantile([0.25, 0.75])
IQR = Q3 - Q1
upper_fence = Q3 + 1.5 * IQR

outliers = df[df['price'] > upper_fence].sort_values('price', ascending=False)
print(f"Upper fence (Q3 + 1.5*IQR): {upper_fence:.0f}")
print(f"\nProperties above this fence ({len(outliers)}):")
print(outliers[['address', 'suburb', 'price', 'beds', 'baths', 'land_size_sqm',
                'property_type']].to_string(index=False))
```

Output:

```
Upper fence (Q3 + 1.5*IQR): 4,646,875
```

Properties above this fence (6):

	address	suburb	price	beds	baths	land_size_sqm
property_type	52A Parriwi Road,	Mosman	12500000	3	4	505.0
House	19 Clanalpine Street,	Mosman	7100000	3	1	752.0
House	49 Raglan Street,	Mosman	6300000	3	2	NaN
House	2/84 Muston Street,	Mosman	5580000	3	2	NaN
Apartment	4/55A Prince Albert Street,	Mosman	5300000	3	2	NaN
Apartment	55 Holt Avenue,	Mosman	4810000	3	1	455.0
House						

Within-suburb outliers:

Code:

```
within_suburb_rows = []
for suburb, group in df.groupby('suburb'):
    q1, q3 = group['price'].quantile([0.25, 0.75])
    fence = q3 + 1.5 * (q3 - q1)
    within_suburb_rows.append(group[group['price'] > fence])

within_suburb = pd.concat(within_suburb_rows)
print(f"Within-suburb outliers ({len(within_suburb)}):")
print(within_suburb[['address', 'suburb', 'price',
'property_type']].to_string(index=False))
```

Output:

Within-suburb outliers (2):

	address	suburb	price	property_type
19	Clanalpine Street,	Mosman	7100000	House
	52A Parriwi Road,	Mosman	12500000	House

Code:

```
print("Median price by suburb:")
print(df.groupby('suburb')['price'].median().sort_values(ascending=False))
print()
print("Correlation between land size and price (houses/semi-detached only, n={}):".format(
    len(df[df['property_type'].isin(['House', 'Semi-detached'])].dropna(subset=['land_size_sqm']))
))
house_like = df[df['property_type'].isin(['House', 'Semi-detached'])].dropna(subset=['land_size_sqm'])
print(house_like[['land_size_sqm', 'price']].corr().iloc[0, 1].round(3))
print()
print("Median price by property type:")
print(df.groupby('property_type')['price'].median().sort_values(ascending=False))
print()
print("Correlation of beds, baths, parking with price (whole dataset):")
print(df[['beds', 'baths', 'parking', 'price']].corr()['price'].drop('price').round(3))
```

Output:

Median price by suburb:

suburb	price
Mosman	3037500.0
Parramatta	815500.0
Campbelltown	628797.5

Name: price, dtype: float64

Correlation between land size and price (houses/semi-detached only, n=20):
-0.061

Median price by property type:

property_type	price
Semi-detached	2950000.0
House	2430000.0
Penthouse	2128500.0
Townhouse	1070000.0

```
Apartment      600000.0  
Name: price, dtype: float64
```

```
Correlation of beds, baths, parking with price (whole dataset):  
beds      0.227  
baths     0.352  
parking    0.531  
Name: price, dtype: float64
```

Part 3: Model Development and Evaluation

Three models were compared on identical cross-validation folds (a single seeded `KFold(n_splits=5)` object reused for all three): Linear Regression, Decision Tree (`max_depth` tuned via `validation_curve`, following the Week 8 module), and KNN (`n_neighbors` tuned via `GridSearchCV`). Scaling for Linear Regression and KNN is fitted separately within each CV fold (using a scikit-learn Pipeline for KNN), not on the full training set beforehand, to avoid leakage between folds. I predicted Linear Regression would win, since suburb is a clean categorical driver; this was not supported.

Decision Tree (`max_depth=5`) achieved the best result on every metric: CV R^2 (0.773 ± 0.178), Test R^2 (0.846), Test RMSE (\$616,121), and Test MAE (\$346,075), and the lowest MAE in every suburb individually. Linear Regression has the largest train-CV gap of the three (0.791 to 0.497), most likely reflecting sensitivity to which rows land in each fold given only 83 rows, rather than genuine overfitting. KNN has the smallest gap (0.777 to 0.545), but this reflects consistently middling fit, not an absence of overfitting risk, its own R^2 values are modest throughout. The large CV standard deviations across all three models (0.157-0.191) show performance depends considerably on fold composition; the Mosman subgroup within the test set is small (9 of 21 rows), so any suburb-specific comparison should be read cautiously. Decision Tree is recommended: it is the clear winner, not one option in a genuine trade-off.

Code:

```
max_depth_range = range(1, 11)

train_scores, val_scores = validation_curve(
    DecisionTreeRegressor(random_state=1), X_train, y_train,
    param_name="max_depth", param_range=max_depth_range,
    cv=cv_strategy, scoring="r2"
)

mean_train_score = np.mean(train_scores, axis=1)
mean_val_score = np.mean(val_scores, axis=1)

plt.figure(figsize=(7, 5))
plt.plot(max_depth_range, mean_train_score, color="blue", label="Training R2")
plt.plot(max_depth_range, mean_val_score, color="red", linestyle="--", label="Cross-
validation R2")
plt.legend(loc="lower right")
plt.xlabel("Tree depth")
plt.ylabel("R2 score")
plt.title("Decision Tree: R2 vs tree depth")
plt.tight_layout()
plt.show()

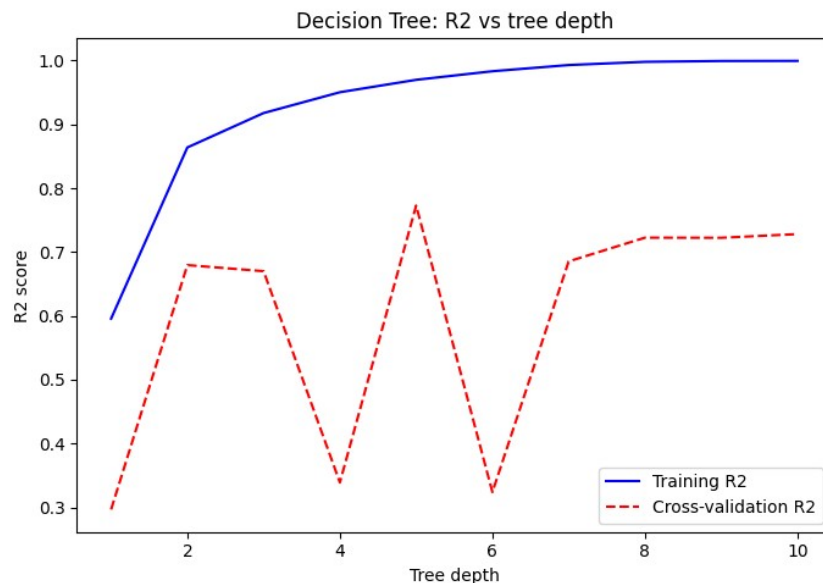
best_depth = max_depth_range[np.argmax(mean_val_score)]
print(f"Best max_depth (by cross-validation R2): {best_depth}")

dt_model = DecisionTreeRegressor(max_depth=best_depth, random_state=1)
dt_cv = cv_metrics(DecisionTreeRegressor(max_depth=best_depth, random_state=1), X_train,
y_train, cv_strategy)

print(f"Decision Tree CV: R2={dt_cv['R2_mean']:.4f} (+/-{dt_cv['R2_std']:.4f}), "
      f"RMSE={dt_cv['RMSE_mean']:.0f} (+/-{dt_cv['RMSE_std']:.0f}), "
      f"MAE={dt_cv['MAE_mean']:.0f} (+/-{dt_cv['MAE_std']:.0f})")
```

Output:

```
Best max_depth (by cross-validation R2): 5
Decision Tree CV: R2=0.7728 (+/-0.1783), RMSE=843,457 (+/-652,196), MAE=404,698 (+/-
214,869)
```



Code:

```
lin_model.fit(X_train_scaled, y_train)
dt_model.fit(X_train, y_train)
knn_model.fit(X_train, y_train)

def full_diagnosis(model, X_tr, y_tr, X_te, y_te, cv):
    train_pred = model.predict(X_tr)
    test_pred = model.predict(X_te)
    return {
        'Train R2': metrics.r2_score(y_tr, train_pred),
        'CV R2 (mean)': cv['R2_mean'], 'CV R2 (std)': cv['R2_std'],
        'Test R2': metrics.r2_score(y_te, test_pred),
        'CV RMSE (mean)': cv['RMSE_mean'], 'CV RMSE (std)': cv['RMSE_std'],
        'Test RMSE': np.sqrt(metrics.mean_squared_error(y_te, test_pred)),
        'CV MAE (mean)': cv['MAE_mean'], 'CV MAE (std)': cv['MAE_std'],
        'Test MAE': metrics.mean_absolute_error(y_te, test_pred),
    }

results = pd.DataFrame({
    'Linear Regression': full_diagnosis(lin_model, X_train_scaled, y_train, X_test_scaled,
y_test, lin_cv),
    'Decision Tree': full_diagnosis(dt_model, X_train, y_train, X_test, y_test, dt_cv),
    'KNN': full_diagnosis(knn_model, X_train, y_train, X_test, y_test, knn_cv),
}).T

print(results.round(4))
```

Output:

	Train R2	CV R2 (mean)	CV R2 (std)	Test R2	\
Linear Regression	0.7912	0.4974	0.1907	0.7475	
Decision Tree	0.9634	0.7728	0.1783	0.8463	
KNN	0.7766	0.5453	0.1570	0.8342	

	CV RMSE (mean)	CV RMSE (std)	Test RMSE	CV MAE (mean)	\
Linear Regression	1.221886e+06	689237.3869	789810.7818	711381.7640	
Decision Tree	8.434568e+05	652195.7793	616120.5993	404697.8843	
KNN	1.117097e+06	563435.1478	639910.1437	532093.5245	

	CV MAE (std)	Test MAE
Linear Regression	283763.9389	561747.6779
Decision Tree	214869.0519	346074.9263
KNN	221142.2395	410809.5238

Per-suburb MAE (test set):

Code:

```
test_suburbs = df.loc[X_test.index, 'suburb']

def per_suburb_mae(model, X_te, y_te, suburbs):
    preds = model.predict(X_te)
```

```

err_df = pd.DataFrame({'suburb': suburbs.values, 'actual': y_te.values, 'predicted':
preds})
err_df['abs_error'] = (err_df['actual'] - err_df['predicted']).abs()
return err_df.groupby('suburb')['abs_error'].agg(['mean', 'count'])

suburb_mae = pd.DataFrame({
    'Linear Regression': per_suburb_mae(lin_model, X_test_scaled, y_test, test_suburbs)
['mean'],
    'Decision Tree': per_suburb_mae(dt_model, X_test, y_test, test_suburbs)['mean'],
    'KNN': per_suburb_mae(knn_model, X_test, y_test, test_suburbs)['mean'],
})
suburb_mae['n (test)'] = per_suburb_mae(dt_model, X_test, y_test, test_suburbs)['count']
print(suburb_mae.round(0))

```

Output:

	Linear Regression	Decision Tree	KNN	n (test)
suburb				
Campbelltown	461243.0	55648.0	232250.0	4
Mosman	867808.0	716926.0	783537.0	9
Parramatta	267682.0	74081.0	80771.0	8

An earlier version of this analysis, before the leakage above was found and fixed, found KNN outperforming Decision Tree specifically on Mosman. That finding did not survive the fix, it was an artefact of scaling statistics being computed from the full training set before cross-validation rather than within each fold.

Decision Tree feature importance:

Code:

```

importances = pd.Series(dt_model.feature_importances_,
index=X_train.columns).sort_values(ascending=False)

plt.figure(figsize=(8, 5))
importances.plot(kind='barh', color='steelblue')
plt.gca().invert_yaxis()
plt.xlabel('Feature importance')
plt.title('Decision Tree feature importance')
plt.tight_layout()
plt.show()

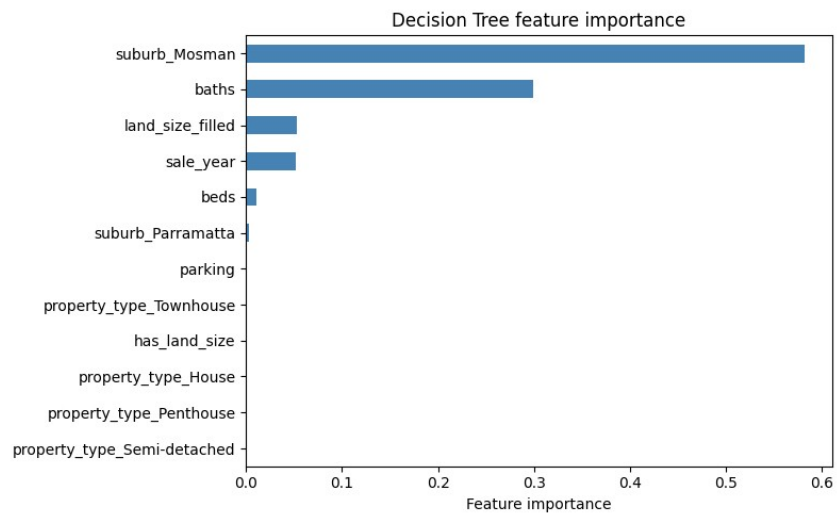
print(importances.round(4))

```

Output:

suburb_Mosman	0.5815
baths	0.2994
land_size_filled	0.0525
sale_year	0.0516
beds	0.0110
suburb_Parramatta	0.0026
parking	0.0010
property_type_Townhouse	0.0004
has_land_size	0.0000
property_type_House	0.0000
property_type_Penthouse	0.0000
property_type_Semi-detached	0.0000

dtype: float64



Part 4: Investigating Prediction Failures

The five largest test-set errors are all Mosman properties (houses, semi-detached, and a townhouse, no apartments), two under-predictions and three over-predictions. Three of the five receive an identical predicted price (\$4,191,250), a direct consequence of single-tree leaf-averaging. Each was examined with recorded features, actual/predicted price, error direction and magnitude, a property-specific explanation grounded in the tree's mechanics, and plausible (not confirmed) missing information such as renovation status or precinct detail.

With only 24 Mosman properties in training, the tree cannot carve out regions fine enough to separate genuinely different properties, nor extrapolate beyond the most expensive training examples it saw. Since none of the worst five are apartments, the evidence more specifically supports "high-value Mosman properties are hard to model" than a general "apartments are harder" claim.

Output:

	address	suburb	property_type	price	predicted_price	abs_error
pct_error						
49	Raglan Street, Mosman	Mosman	House	\$6,300,000	\$4,191,250	\$2,108,750
33.5%						
23	Holt Avenue, Mosman	Mosman	House	\$4,275,000	\$3,098,917	\$1,176,083
27.5%						
93	Holt Avenue, Mosman	Mosman	Semi-detached	\$3,075,000	\$4,191,250	\$1,116,250
36.3%						
45	Myahgah Road, Mosman	Mosman	Semi-detached	\$3,660,000	\$4,191,250	\$531,250
14.5%						
3/12	Earl Street, Mosman	Mosman	Townhouse	\$2,900,000	\$2,410,000	\$490,000
16.9%						

Part 5: Deployment

The dataset contains 104 properties; the deployed model was trained on the same 83-row training subset used throughout Part 3, with 21 rows held out for testing. The final model (Decision Tree, max_depth=5, 8 features after excluding total_rooms) was exported with joblib as four files: model.joblib, model_columns.joblib, suburb_options.joblib, and property_type_options.joblib. The Streamlit app (app.py) applies identical encoding to training and no scaling (unnecessary for a tree). Land size is entered via an explicit Unknown/Known choice rather than a default number. Input fields are constrained to the exact ranges observed in training (bedrooms 1-6, bathrooms 1-4, parking 0-8, land size 74-1233 sqm when known, sale year 2017-2026) to avoid unsupported extrapolation, alongside a point-estimate warning and a Mosman-specific caution.

Deployment trade-off: a single tree is fast, needs no scaling, and is directly auditable, but produces stepwise predictions (Part 4) and, with only 83 training rows, is less stable than an ensemble would be. To run: pip install streamlit joblib pandas scikit-learn, then streamlit run app.py.

Application screenshots (verified live predictions):

This screenshot shows the default state of the Streamlit app. The 'Suburb' dropdown is set to 'Campbelltown' and 'Property type' is set to 'Apartment'. The 'Bedrooms' field is 3, 'Bathrooms' is 2, 'Parking spaces' is 1, and 'Sale year' is 2026. The 'Land size' dropdown is set to 'Unknown / not applicable (typical for apartments/units)'. A 'Predict price' button is at the bottom.

Step 1: default form state, Campbelltown apartment, land size left Unknown.

This screenshot shows the app after clicking the 'Predict price' button. The predicted price is displayed at the bottom in blue text: '\$692,917'. A small downward arrow icon is visible next to the price.

Step 2: Campbelltown inputs predict \$692,917, verified against the Python model exactly.

This screenshot shows the app with the 'Suburb' dropdown changed to 'Mosman'. All other inputs remain the same: 'Property type' is 'Apartment', 'Bedrooms' is 2, 'Bathrooms' is 2, 'Parking spaces' is 1, 'Sale year' is 2026, and 'Land size' is 'Unknown / not applicable (typical for apartments/units)'. The 'Predict price' button is at the bottom.

Step 3: changing only suburb to Mosman (same property profile) predicts \$2,410,000, also verified exactly, a direct illustration of suburb as the dominant price driver.

Final Reflection

The most significant lesson was how much effort real-world data collection takes. Domain.com.au's inconsistent price disclosure shaped the whole project before modelling began, and revealed two distinct selection-bias mechanisms: listings that disclose a price are not a random sample, and within Mosman specifically, disclosure skewed toward larger homes.

Feature engineering produced two surprises: land size showed almost no correlation with price among houses, and a derived feature (`total_rooms`) turned out to be an exact redundant combination of two included features.

The single most important technical lesson came from finding and fixing a genuine data leakage bug: scaling statistics for Linear Regression and KNN were originally computed once from the full training set before cross-validation, rather than separately within each fold. Fixing it changed KNN's selected hyperparameter and reversed an entire conclusion, an earlier version of this analysis found KNN outperformed Decision Tree specifically on Mosman; once leakage was removed, Decision Tree outperformed KNN on every metric in every suburb. A subtle preprocessing bug had produced a plausible, defensible-sounding finding that was actually an artefact of the bug itself. I will now always check exactly when and on what data a preprocessing statistic is computed, especially inside a cross-validation loop.

The clearest fairness concern is that this model is least reliable exactly where accuracy matters most, unique, high-value Mosman properties, since 24 training examples cannot represent that suburb's true diversity. With more time, I would collect substantially more Mosman data, add precinct-level location detail, and compare against an ensemble method.

GitHub Repository

Full dataset, notebook, application source code (`app.py`, `model.joblib`, `model_columns.joblib`, `suburb_options.joblib`, `property_type_options.joblib`), and README with run instructions: [INSERT YOUR GITHUB OR ONEDRIVE/DROPBOX LINK HERE]

References

Domain Holdings Australia. (2026). Domain: Real estate & property for sale. <https://www.domain.com.au>

NSW Valuer General. (2026). Property sales data. NSW Government. <https://www.valuergeneral.nsw.gov.au>

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Streamlit Inc. (2026). Streamlit documentation. <https://docs.streamlit.io>

Acknowledgement of GenAI Use

Generative AI was used to help brainstorm ideas during the analysis and to tighten up my own writing across the report sections, after I had drafted my initial ideas, code, and explanations myself.